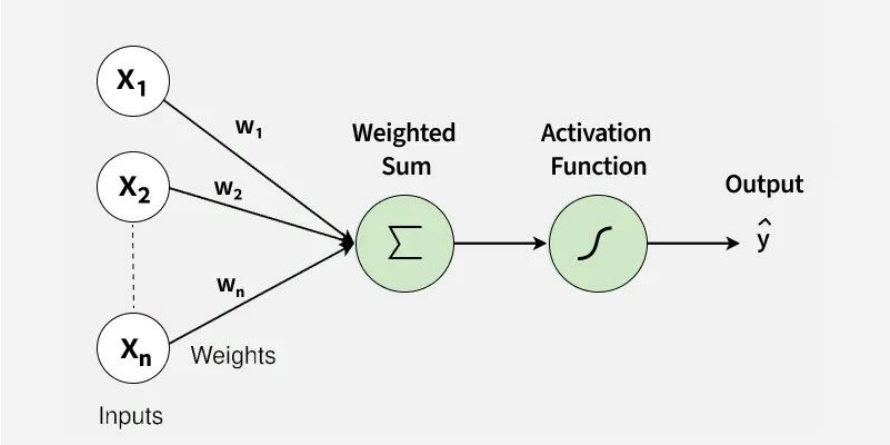
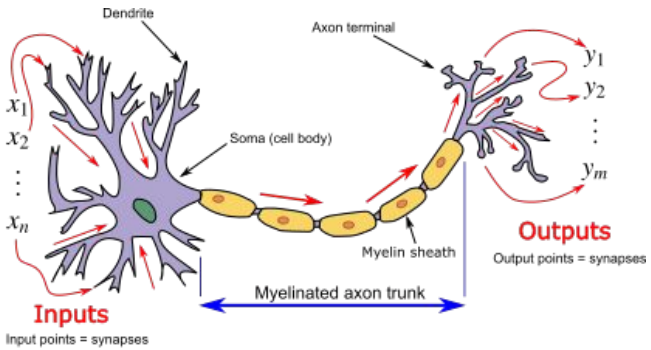
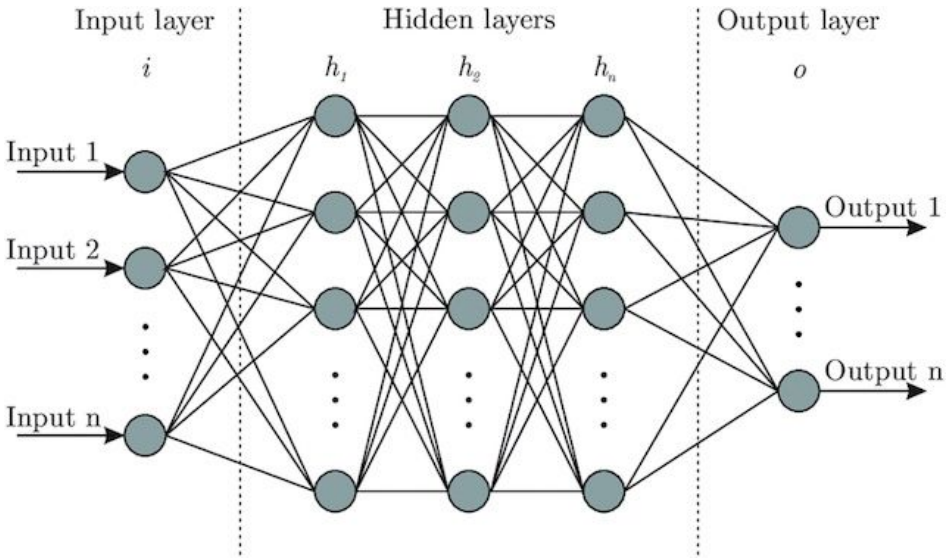
# From theory to PyTorch: neural network basics

CH-457

# Neuron to neural network

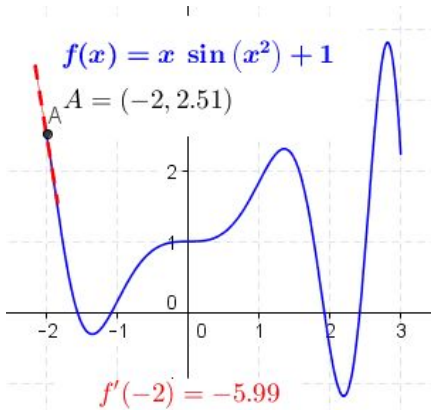


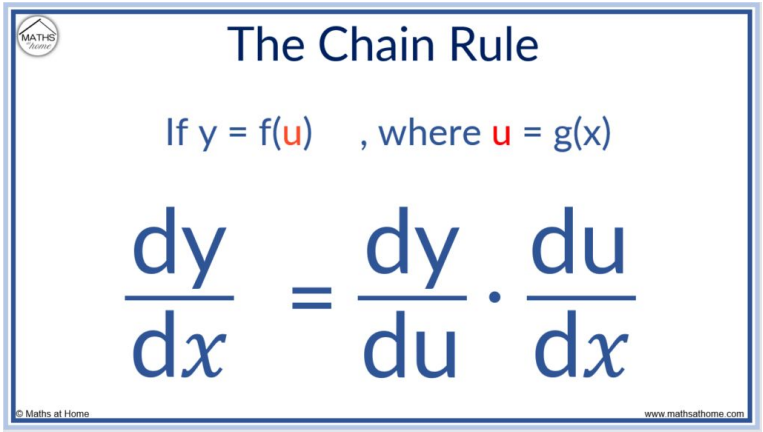



# Calculus review: derivative and gradients

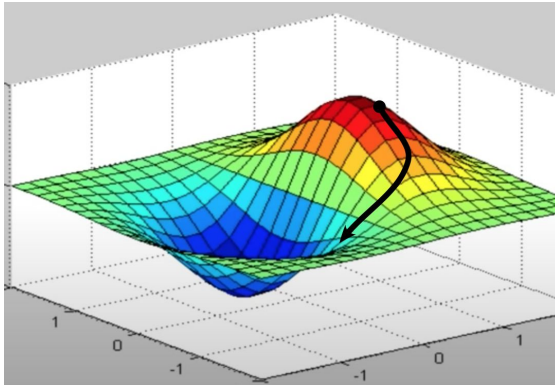
- The derivative is a **value** that quantifies the rate of change of a functions output wrt its input **(scalar)**
- The gradient of a multivariate function is a **vector**-valued function whose value at a point gives the direction and rate of fastest increase **(vector)**

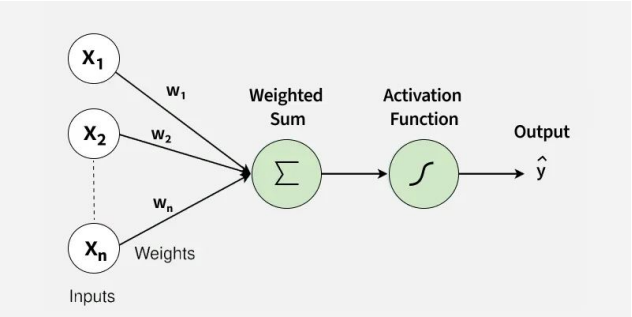


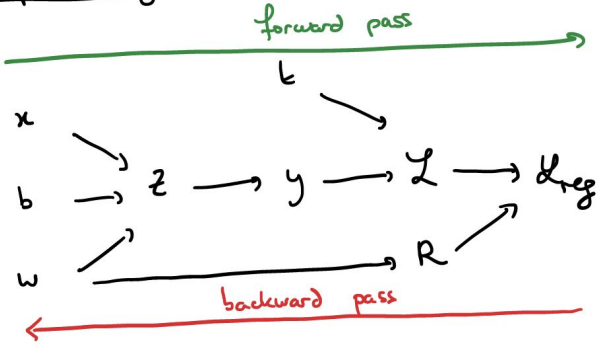
# Putting it all together

We are trying to learn a function that models our data by optimizing a loss function




Computation graph




$z = wx + b$

$y = \sigma(z)$

$\mathcal{L} = \frac{1}{2}(y-t)^2$

$R = \frac{1}{2}w^2$

$$\boxed{\mathcal{L}_{reg} = \mathcal{L} + \lambda R}$$

Goal: Compute $\frac{\partial \mathcal{L}_{reg}}{\partial w}$ and $\frac{\partial \mathcal{L}_{reg}}{\partial b}$

equivalent $\bar{w}$ and $\bar{b}$ since $\bar{w} = \frac{\partial \mathcal{L}_{reg}}{\partial w}$

$\bar{b} = \frac{\partial \mathcal{L}_{reg}}{\partial b}$

# Putting it all together: backpropagation

Linear regression

Example

$$\bar{z} = \bar{y}\frac{dy}{dz} = \bar{y}\sigma'(z)$$

$$= \bar{\mathcal{L}}(y-t)$$

$$\bar{y} = \bar{\mathcal{L}}\frac{d\mathcal{L}}{dy}$$

$$\bar{\mathcal{L}} = \bar{\mathcal{L}_{reg}}\frac{d\mathcal{L}_{reg}}{d\mathcal{L}} = \bar{\mathcal{L}_{reg}}$$

$$\boxed{\bar{b} = \bar{z}\frac{dz}{db} = \bar{z}}$$

$$x \quad b \quad w \quad t$$

$$z = wx + b \longrightarrow y = \sigma(z) \longrightarrow \mathcal{L} = \frac{1}{2}(y-t)^2$$

$$\mathcal{L}_{reg} = \mathcal{L} + \lambda R$$

$$w \longrightarrow R = \frac{1}{2}w^2$$

$$\bar{\mathcal{L}_{reg}} = \frac{d\mathcal{L}_{reg}}{d\mathcal{L}_{reg}} = 1$$

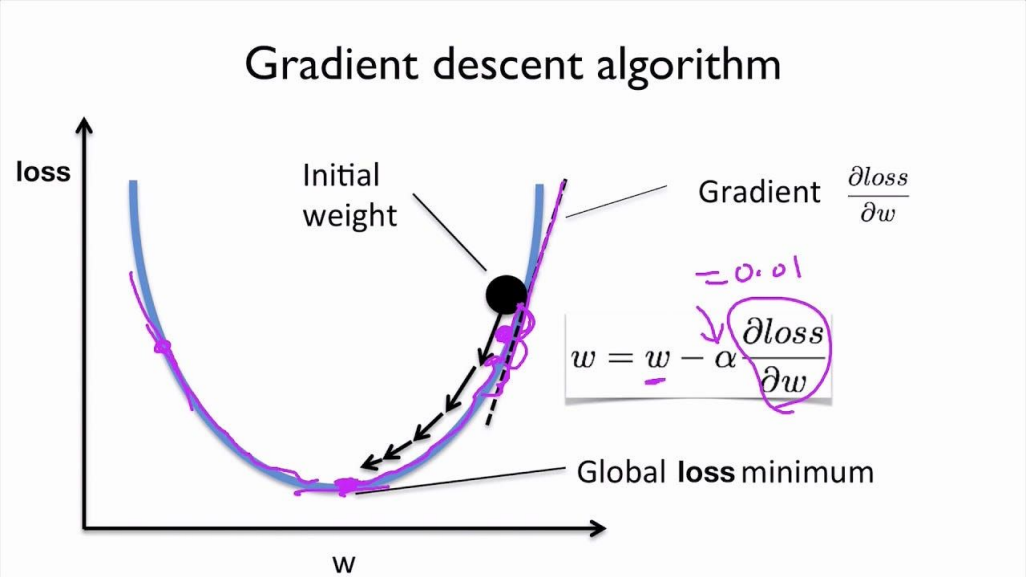
$$\bar{w} = \bar{z}\frac{dz}{dw} + \bar{R}\frac{dR}{dw}$$

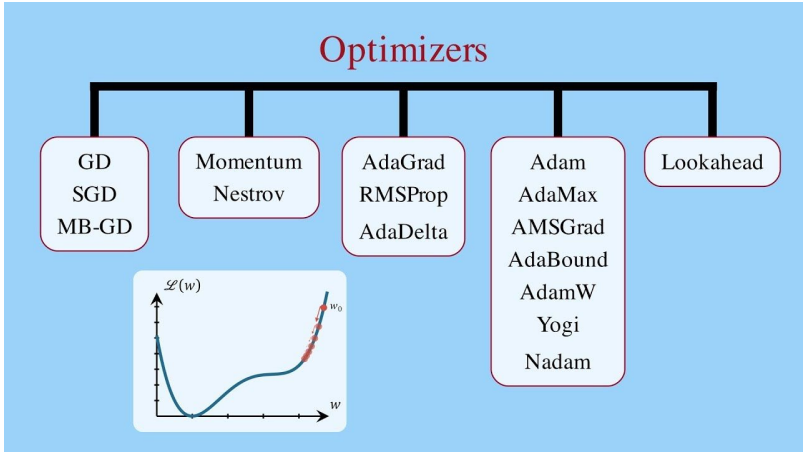
$$\bar{R} = \bar{\mathcal{L}_{reg}}\frac{d\mathcal{L}_{reg}}{dR}$$

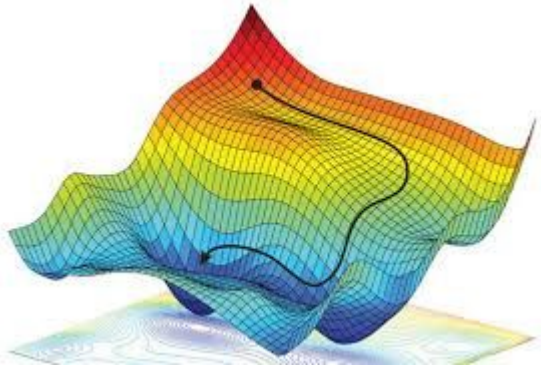
$$= \bar{\mathcal{L}_{reg}}\lambda$$

$$\boxed{\bar{w} = \bar{z}x + \bar{R}w}$$

# Optimizers


Gradient descent algorithm
loss
Initial weight
Gradient $\frac{\partial loss}{\partial w}$
$= 0.01$
$w = w - \alpha \frac{\partial loss}{\partial w}$
Global loss minimum
w

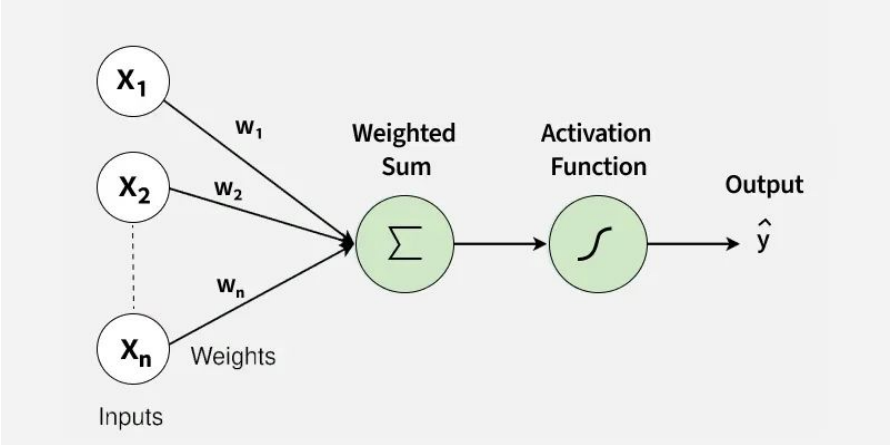

Optimizers
GD
SGD
MB-GD
Momentum
Nestrov
AdaGrad
RMSProp
AdaDelta
Adam
AdaMax
AMSGrad
AdaBound
AdamW
Yogi
Nadam
Lookahead
$\mathcal{L}(w)$
$w_0$
$w$



# What this looks like in Pytorch: the network

```python
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```



# What this looks like in Pytorch: the data

```
import os
import pandas as pd
from torchvision.io import decode_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = decode_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

# What this looks like in Pytorch: the data

```python
import os
import pandas as pd
from torchvision.io import decode_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = transform
    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = decode_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

```python
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```

# What this looks like in Pytorch: training

```
net = Net()
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

```
for epoch in range(2):  # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0

print('Finished Training')
```

# Nice tutorials / further reading

- https://sebastianraschka.com/teaching/pytorch-1h/#1-what-is-pytorch
- https://www.youtube.com/watch?v=VMj-3S1tku0